Lecture 1: Algorithm Analysis and Big-O Notation

An algorithm is a finite sequence of well-defined steps that solves a problem. To compare algorithms independently of hardware, we count how the number of basic operations grows as the input size n grows. This growth rate is called the time complexity of the algorithm.

Big-O notation describes an upper bound on growth. We say f(n) is O(g(n)) if there are constants c > 0 and n0 such that f(n) <= c * g(n) for every n >= n0. Constant factors and lower-order terms are ignored, so 3n^2 + 5n + 7 is O(n^2).

Common complexity classes, from fastest to slowest growing, are: O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n) linearithmic, O(n^2) quadratic, O(2^n) exponential and O(n!) factorial. Binary search is O(log n) because it halves the search range on every step.

Big-Omega gives a lower bound and Big-Theta gives a tight bound, meaning the function is bounded both above and below by the same growth rate.

Best-case, average-case and worst-case analysis describe performance on different inputs. Linear search has a best case of O(1) when the target is the first element and a worst case of O(n) when the target is missing. Unless stated otherwise, complexity in this course refers to the worst case.

Space complexity measures the extra memory an algorithm needs in addition to its input. An in-place algorithm uses only O(1) extra space.

Amortised analysis averages the cost of a sequence of operations. Appending to a dynamic array is O(1) amortised: most appends are cheap, and the occasional resize that copies every element is rare enough that its cost is spread across many appends.

Lecture 2: Arrays and Linked Lists

An array stores elements in one contiguous block of memory. Because the address of element i can be computed directly, reading or writing any index takes O(1) time. This property is called random access.

Inserting or deleting in the middle of an array is O(n), because every later element has to be shifted by one position. A static array has a fixed capacity chosen when it is created.

A dynamic array, such as a Python list or a Java ArrayList, grows automatically. When it runs out of space it allocates a new block, usually double the old capacity, and copies the elements across. Doubling is what makes appends O(1) amortised.

A singly linked list stores each element in a node together with a pointer to the next node. The list keeps a reference to the head node. Inserting or deleting at the head is O(1), but accessing the k-th element requires walking from the head, which is O(n).

A doubly linked list also stores a pointer to the previous node. This makes deleting a node O(1) when we already hold a reference to it, and allows traversal in both directions, at the cost of extra memory per node.

Arrays have better cache locality than linked lists because neighbouring elements sit next to each other in memory. In practice this often makes arrays faster even when the asymptotic complexity is the same.

Use an array when you need fast indexed access and mostly append at the end. Use a linked list when you frequently insert or remove elements at known positions and rarely need random access.

Lecture 3: Stacks and Queues

A stack is a last-in, first-out (LIFO) collection. The two main operations are push, which adds an element to the top, and pop, which removes the most recently added element. Peek returns the top element without removing it. All three operations are O(1).

Stacks are used for function call management (the call stack), undo features in editors, checking balanced parentheses, and evaluating expressions written in postfix notation. Depth-first search can be implemented with an explicit stack instead of recursion.

A queue is a first-in, first-out (FIFO) collection. Enqueue adds an element to the back and dequeue removes the element at the front. Both operations are O(1) when the queue is implemented as a linked list or as a circular buffer.

A circular buffer stores a queue in a fixed-size array and keeps two indices, front and rear, that wrap around to the start of the array when they reach the end. This avoids shifting elements after every dequeue.

Queues are used for scheduling tasks, buffering data between a producer and a consumer, and breadth-first search, which visits graph vertices in order of their distance from the start.

A deque (double-ended queue) supports insertion and removal at both ends in O(1). A priority queue removes the element with the highest priority rather than the oldest one. Priority queues are usually implemented with a binary heap, giving O(log n) insert and remove-min.

Lecture 4: Hash Tables

A hash table stores key-value pairs and supports insert, lookup and delete in O(1) time on average. A hash function maps each key to an integer, and that integer modulo the table size gives the index of a bucket.

A good hash function is deterministic, fast to compute and spreads keys uniformly across the buckets. When two different keys map to the same bucket, a collision occurs. Collisions are unavoidable because there are more possible keys than buckets.

Separate chaining resolves collisions by storing a linked list (or another small collection) in each bucket. All keys that hash to the same index are kept in that bucket's chain.

Open addressing stores every entry directly in the array. When a bucket is taken, the table probes other positions. Linear probing checks the next slot, quadratic probing jumps by increasing square offsets, and double hashing uses a second hash function to choose the step.

The load factor is the number of stored entries divided by the number of buckets. As the load factor grows, collisions become more frequent and operations slow down. Most implementations resize the table, typically doubling it and rehashing every key, when the load factor exceeds a threshold such as 0.75.

In the worst case, when every key collides, hash table operations degrade to O(n). Hash tables do not keep keys in sorted order; if ordered traversal is needed, a balanced search tree is a better choice.

Lecture 5: Trees and Binary Search Trees

A tree is a hierarchical structure made of nodes connected by edges, with a single root node and no cycles. A node with no children is a leaf. The height of a tree is the number of edges on the longest path from the root to a leaf.

In a binary tree each node has at most two children, called left and right. A binary search tree (BST) adds an ordering rule: every key in the left subtree is smaller than the node's key and every key in the right subtree is larger.

Search, insert and delete in a BST take O(h) time, where h is the height. If the tree is balanced, h is O(log n). If keys are inserted in sorted order, the tree degenerates into a linked list and h becomes O(n).

Self-balancing trees such as AVL trees and red-black trees perform rotations after inserts and deletes to keep the height O(log n). An AVL tree keeps the heights of the two subtrees of every node within one of each other.

Tree traversals visit every node once. Pre-order visits the node before its subtrees, in-order visits the left subtree, then the node, then the right subtree, and post-order visits the node after both subtrees. An in-order traversal of a BST returns the keys in sorted order. Level-order traversal visits nodes level by level using a queue.

A binary heap is a complete binary tree where every parent is smaller than its children in a min-heap. Heaps are stored compactly in arrays: the children of index i are at 2i + 1 and 2i + 2.

Lecture 6: Graphs, BFS and DFS

A graph G = (V, E) consists of a set of vertices V and a set of edges E. In a directed graph each edge has a direction, and in a weighted graph each edge carries a numeric weight such as a distance or a cost.

An adjacency list stores, for every vertex, the list of its neighbours. It uses O(V + E) memory and suits sparse graphs. An adjacency matrix is a V x V table where entry (u, v) shows whether an edge exists. It uses O(V^2) memory but answers "is there an edge from u to v?" in O(1), which suits dense graphs.

Breadth-first search (BFS) explores the graph level by level from a start vertex using a queue. In an unweighted graph, BFS finds the shortest path, measured in number of edges, from the start to every reachable vertex.

Depth-first search (DFS) follows one path as far as possible before backtracking, using recursion or an explicit stack. DFS is used for detecting cycles, finding connected components, and topological sorting of directed acyclic graphs.

Both BFS and DFS run in O(V + E) time with an adjacency list, because every vertex and every edge is processed a constant number of times. Both must mark visited vertices to avoid infinite loops in graphs with cycles.

For weighted graphs with non-negative weights, Dijkstra's algorithm finds shortest paths using a priority queue in O((V + E) log V) time. When edge weights can be negative, the Bellman-Ford algorithm is used instead.

Lecture 7: Sorting Algorithms

A sorting algorithm arranges elements in order. A sort is stable if elements with equal keys keep their original relative order. Comparison-based sorting has a lower bound of O(n log n) comparisons in the worst case.

Bubble sort repeatedly swaps adjacent elements that are out of order. Insertion sort builds a sorted prefix by inserting each new element into its correct position. Both are O(n^2) in the worst case, but insertion sort is O(n) on nearly sorted input and is often used for small arrays.

Merge sort is a divide-and-conquer algorithm. It splits the array in half, recursively sorts each half, and merges the two sorted halves. Merge sort always runs in O(n log n) time and is stable, but it needs O(n) extra memory for merging.

Quicksort picks a pivot, partitions the array so that smaller elements come before the pivot and larger ones after it, and then recursively sorts both parts. Its average running time is O(n log n), but a consistently bad pivot, for example always the smallest element, gives the O(n^2) worst case. Choosing a random pivot makes the worst case very unlikely. Quicksort sorts in place and is usually fast in practice, but it is not stable.

Heapsort builds a max-heap and repeatedly moves the largest element to the end. It runs in O(n log n) time with O(1) extra space, but it is not stable.

Counting sort and radix sort are not comparison-based. They can sort integers in linear time when the range of key values is limited.

Lecture 8: Recursion and Dynamic Programming

A recursive function solves a problem by calling itself on smaller instances of the same problem. Every recursive function needs a base case that stops the recursion; without one the calls continue until the program runs out of stack space and raises a stack overflow error.

The naive recursive Fibonacci function, fib(n) = fib(n - 1) + fib(n - 2), runs in exponential time because it recomputes the same subproblems many times.

Dynamic programming (DP) applies when a problem has two properties: optimal substructure, where an optimal solution can be built from optimal solutions of subproblems, and overlapping subproblems, where the same subproblems appear again and again.

Memoisation is the top-down approach: keep the recursive structure but store each computed result in a cache, so every subproblem is solved only once. Tabulation is the bottom-up approach: fill a table iteratively, starting from the smallest subproblems. With either approach, Fibonacci becomes O(n).

Classic DP problems include the 0/1 knapsack problem, longest common subsequence, edit distance between two strings, and the coin change problem. Solving a DP problem usually starts by defining the state, writing the recurrence relation between states, and identifying the base cases.

Greedy algorithms make the locally best choice at each step and never reconsider it. They are simpler and faster than DP, but they only give an optimal answer for problems with the greedy choice property, such as making change with standard coin denominations or building a minimum spanning tree.